

Q2 Bank Account Transaction Management

CODE:

```
import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.PreparedStatement;

import java.sql.ResultSet;

import java.sql.SQLException;

import java.sql.Statement;

import java.util.Scanner;


public class BankAccountTransactionManagement {

    // Added allowPublicKeyRetrieval=true to resolve the Public Key Retrieval error

    private static final String URL =
"jdbc:mysql://localhost:3306/bankdb?createDatabaseIfNotExist=true&useSSL=false&allowPubl
icKeyRetrieval=true";

    private static final String USER = "root";

    private static final String PASSWORD = "Abhay@12345"; // Update if your MySQL
password is empty ("" ) or different


    // Initialize database table if it doesn't already exist

    public static void initializeDatabase(Connection conn) {

        String sql = "CREATE TABLE IF NOT EXISTS accounts (" +

            "account_number INT PRIMARY KEY, " +

            "account_holder_name VARCHAR(100) NOT NULL, " +

            "balance DOUBLE NOT NULL)";

        try (Statement stmt = conn.createStatement()) {
```

```

        stmt.execute(sql);
    } catch (SQLException e) {
        System.err.println("Database initialization error: " + e.getMessage());
    }
}

```

// Create a new bank account

```

public static void createAccount(Connection conn, int accNo, String name, double balance) {
    String sql = "INSERT INTO accounts (account_number, account_holder_name, balance)
VALUES (?, ?, ?)";

    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setInt(1, accNo);
        pstmt.setString(2, name);
        pstmt.setDouble(3, balance);
        pstmt.executeUpdate();

        System.out.println("Account created successfully. Balance = " + (int) balance);
    } catch (SQLException e) {
        System.err.println("Error creating account: " + e.getMessage());
    }
}

```

// Deposit money into an account

```

public static void deposit(Connection conn, int accNo, double amount) {
    String updateSql = "UPDATE accounts SET balance = balance + ? WHERE
account_number = ?";

    try (PreparedStatement pstmt = conn.prepareStatement(updateSql)) {

```

```

        pstmt.setDouble(1, amount);

        pstmt.setInt(2, accNo);

        int rowsUpdated = pstmt.executeUpdate();

        if (rowsUpdated > 0) {

            displayBalance(conn, accNo);

        } else {

            System.out.println("Account not found.");

        }

    } catch (SQLException e) {

        System.err.println("Deposit transaction failed: " + e.getMessage());

    }

}

```

// Withdraw money with balance validation

```

public static void withdraw(Connection conn, int accNo, double amount) {

    String selectSql = "SELECT balance FROM accounts WHERE account_number = ?";

    String updateSql = "UPDATE accounts SET balance = balance - ? WHERE
account_number = ?";

    try (PreparedStatement checkStmt = conn.prepareStatement(selectSql)) {

        checkStmt.setInt(1, accNo);

        try (ResultSet rs = checkStmt.executeQuery()) {

            if (rs.next()) {

                double currentBalance = rs.getDouble("balance");

                if (amount > currentBalance) {

```

```

        System.out.println("Insufficient Balance");
        return;
    }

    try (PreparedStatement updateStmt = conn.prepareStatement(updateSql)) {
        updateStmt.setDouble(1, amount);
        updateStmt.setInt(2, accNo);
        updateStmt.executeUpdate();
        displayBalance(conn, accNo);
    }
} else {
    System.out.println("Account not found.");
}
}

} catch (SQLException e) {
    System.err.println("Withdrawal transaction failed: " + e.getMessage());
}
}

// Fetch and display account details / balance matching test cases
public static void displayBalance(Connection conn, int accNo) {
    String sql = "SELECT balance FROM accounts WHERE account_number = ?";
    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setInt(1, accNo);
        try (ResultSet rs = pstmt.executeQuery()) {

```

```

        if (rs.next()) {

            double balance = rs.getDouble("balance");

            System.out.println("Balance = " + (int) balance);

        }

    }

} catch (SQLException e) {

    System.err.println("Error fetching balance: " + e.getMessage());

}

}

public static void main(String[] args) {

    try {

        Class.forName("com.mysql.cj.jdbc.Driver");

    } catch (ClassNotFoundException e) {

        try {

            Class.forName("com.mysql.jdbc.Driver");

        } catch (ClassNotFoundException ignored) {}

    }

    try (Connection conn = DriverManager.getConnection(URL, USER, PASSWORD);

        Scanner sc = new Scanner(System.in)) {

        initializeDatabase(conn);

        while (true) {

```

```
System.out.println("\n=== Bank Transaction System ===");

System.out.println("1. Create Account");

System.out.println("2. Deposit Money");

System.out.println("3. Withdraw Money");

System.out.println("4. Display Account Balance");

System.out.println("5. Exit");

System.out.print("Choose an option: ");

if (!sc.hasNextLine()) break;

String choice = sc.nextLine().trim();

switch (choice) {

    case "1":

        System.out.print("Enter Account Number: ");

        int accNo = Integer.parseInt(sc.nextLine().trim());

        System.out.print("Enter Account Holder Name: ");

        String name = sc.nextLine().trim();

        System.out.print("Enter Initial Balance: ");

        double balance = Double.parseDouble(sc.nextLine().trim());

        createAccount(conn, accNo, name, balance);

        break;

    case "2":

        System.out.print("Enter Account Number: ");

        int depAcc = Integer.parseInt(sc.nextLine().trim());
```

```
System.out.print("Enter Deposit Amount: ");  
  
double depAmount = Double.parseDouble(sc.nextLine().trim());  
  
deposit(conn, depAcc, depAmount);  
  
break;
```

case "3":

```
System.out.print("Enter Account Number: ");  
  
int withAcc = Integer.parseInt(sc.nextLine().trim());  
  
System.out.print("Enter Withdrawal Amount: ");  
  
double withAmount = Double.parseDouble(sc.nextLine().trim());  
  
withdraw(conn, withAcc, withAmount);  
  
break;
```

case "4":

```
System.out.print("Enter Account Number: ");  
  
int dispAcc = Integer.parseInt(sc.nextLine().trim());  
  
displayBalance(conn, dispAcc);  
  
break;
```

case "5":

```
System.out.println("Exiting application.");  
  
return;
```

default:

```
System.out.println("Invalid option. Please try again.");
```

```
    }  
    }  
    } catch (SQLException e) {  
        System.err.println("Database connection failed: " + e.getMessage());  
    }  
    }  
}
```



```
C:\Windows\System32\cmd.e X + v

=== Bank Transaction System ===
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Display Account Balance
5. Exit
Choose an option: 1
Enter Account Number: 1001
Enter Account Holder Name: Arun
Enter Initial Balance: 10000
Account created successfully. Balance = 10000

=== Bank Transaction System ===
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Display Account Balance
5. Exit
Choose an option: 2
Enter Account Number: 1001
Enter Deposit Amount: 2000
Balance = 12000

=== Bank Transaction System ===
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Display Account Balance
5. Exit
Choose an option: 4
Enter Account Number: 1001
Balance = 12000

=== Bank Transaction System ===
1. Create Account
```

```
C:\Windows\System32\cmd.e  X + v
Enter Account Number: 1001
Balance = 12000

=== Bank Transaction System ===
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Display Account Balance
5. Exit
Choose an option: 3
Enter Account Number: 1001
Enter Withdrawal Amount: 3000
Balance = 9000

=== Bank Transaction System ===
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Display Account Balance
5. Exit
Choose an option: 4
Enter Account Number: 1001
Balance = 9000

=== Bank Transaction System ===
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Display Account Balance
5. Exit
Choose an option: 3
Enter Account Number: 1001
Enter Withdrawal Amount: 15000
Insufficient Balance
```